

nginx JS — Dynamic Reconfiguration Guide (locations)

Table of Contents

1. Overview	2
1.1. Scope	3
1.2. Config phase vs request phase	3
1.3. nginx location matching algorithm	3
2. Inspecting the location tree	4
2.1. <code>srv.locations[]</code>	4
2.2. <code>srv.findLocation(pattern)</code>	4
2.3. Iterating and filtering	5
3. Adding locations	5
3.1. <code>srv.addLocation(pattern [, opts])</code>	5
3.2. <code>loc.addLocation(pattern [, opts])</code>	6
3.3. Example: add a simple prefix location	6
3.4. Example: add an exact-match health endpoint	7
3.5. Example: add a regex location at a specific position	7
3.6. Example: add a named location for error fallback	7
3.7. Example: add a location using a template	8
3.8. Flow diagram: <code>addLocation</code> BST rebuild	8
4. Removing locations	8
4.1. <code>srv.removeLocation(pattern)</code>	8
4.2. Example: maintenance mode toggle	9
4.3. Example: remove all dynamically added locations	10
4.4. Flow diagram: <code>removeLocation</code> BST rebuild	10
5. Cloning locations	11
5.1. <code>loc.clone(newPattern [, opts])</code>	11
5.2. Example: versioned API clone	11
5.3. Example: full subtree clone with <code>depth:Infinity</code>	12
5.4. Example: clone a single location, no children	12
6. Cloning a server	12
6.1. <code>srv.clone(newServerName)</code>	12
6.2. Example: per-tenant server provisioning	13
7. Modifying existing locations	13
7.1. Changing handler and properties	13
7.2. Atomic batch modification	13
7.3. Rollback with snapshot	14

7.4. Location swap (atomic replace)	14
8. Atomic location changes across all workers	15
8.1. The problem	15
8.2. Pattern: broadcast structural change	15
8.3. Flow diagram: cross-worker addLocation	16
9. Safe patterns	16
9.1. Idempotent add-or-update	16
9.2. Temporary location (feature flag)	17
9.3. Versioned API routing	17
9.4. Location inventory endpoint	17
9.5. A/B routing by location clone	18
10. API Reference	18
10.1. <code>srv.locations</code>	18
10.2. <code>srv.findLocation(pattern)</code>	19
10.3. <code>srv.addLocation(pattern [, opts])</code>	19
10.4. <code>srv.removeLocation(pattern)</code>	19
10.5. <code>srv.clone(newServerName)</code>	19
10.6. <code>loc.addLocation(pattern [, opts])</code>	19
10.7. <code>loc.removeLocation(pattern)</code>	20
10.8. <code>loc.clone(newPattern [, opts])</code>	20
10.9. <code>loc.path</code> , <code>loc.matchType</code> , <code>loc.pattern</code> , <code>loc.internal</code>	20
10.10. <code>nginx.suspendAcceptance()</code> / <code>nginx.resumeAcceptance()</code>	20
10.11. <code>nginx.suspendAllWorkers()</code> / <code>nginx.resumeAllWorkers()</code>	21
11. Design notes	21
11.1. BST rebuild cost	21
11.2. Dynamic pool vs static pool	21
11.3. <code>r.location</code> is structural read-only	21
11.4. Regex position and definition order	21
11.5. Handler serialisation for cross-worker broadcast	22



DRAFT FOR COMMENTS — This document is a work in progress. Content, API names, and examples are subject to change before release.

1. Overview

This guide covers runtime management of nginx **location blocks** via the JavaScript DOM: adding, removing, cloning, and reconfiguring locations and servers without reloading nginx.

Prerequisites: read the *for single worker* guide first for `write_mode`, `snapshot()`, and `suspendAcceptance()`.

1.1. Scope

Location operations are available on:

- `NginxServer` objects — `nginx.http.servers[i]`
- `NginxLocation` objects — `nginx.http.servers[i].locations[j]` or the return value of `addLocation()`

They are **not** available on `req.location` (the request-phase wrapper is read-only for structural operations). Field writes (`root`, `handler`, ...) work on `req.location` subject to `write_mode`.

1.2. Config phase vs request phase

Location structure mutations (add, remove, clone) and handler assignment are permanent, global changes — they update the location BST immediately and affect every subsequent request on this worker. There is no per-request isolation for structural changes.

To propagate structural changes to all workers, use a `SharedWorker` broadcast combined with `suspendAllWorkers()` (see *for all workers* guide).

1.3. nginx location matching algorithm

Understanding the matching order is essential when adding locations:

```
Incoming URI
|
[]
1. Exact-match locations (= /path) ← checked first; wins immediately on match
|
[]
2. Prefix locations (longest wins among all prefix matches)
   |
   |— [] If longest match has ^~ ← skip regex scan; use this location
   |— [] Otherwise: remember the longest-prefix winner and continue ...
|
[]
3. Regex locations (checked in definition order, first match wins)
   |
   |— [] Match found ← use this location (overrides prefix winner)
   |— [] No match ← fall back to the remembered prefix winner
|
[]
4. Named locations (@name) – reachable only from error_page / try_files
```

This order determines which `addLocation()` patterns affect which requests.

2. Inspecting the location tree

2.1. `srv.locations[]`

Returns a live array of all locations registered on this server, including dynamically added ones. The order is: prefix/exact locations (sorted by path length, longest first), then regex locations (definition order), then named locations.

```
const srv = nginx.http.servers[0];

// List all locations with their match type and path
srv.locations.forEach(loc => {
  nginx.log(`${loc.matchType.padEnd(16)} ${loc.path}`);
});
```

`loc.matchType` values:

Value	Meaning
'prefix'	Normal longest-prefix match.
'exact'	<code>= /path</code> — checked first, wins immediately.
'prefix_noregex'	<code>^~ /path</code> — skips regex scan on match.
'regex'	<code>~ /pattern</code> — case-sensitive PCRE.
'regex_caseless'	<code>~* /pattern</code> — case-insensitive PCRE.
'named'	<code>@name</code> — reachable from <code>error_page</code> / <code>try_files</code> only.

`loc.path` — the full pattern string as passed to `addLocation()`, including the modifier prefix (e.g. `"=/exact"`, `"~ /\.\.php$"`).

`loc.pattern` — the bare path/regex without the modifier prefix.

`loc.internal` — `true` for named and noname (`limit_except`, etc.) locations.

2.2. `srv.findLocation(pattern)`

Searches the server's location arrays by exact pattern string and returns the `NginxLocation` wrapper, or `null` if not found.

```
const srv = nginx.http.servers[0];

const api = srv.findLocation('/api/');
if (api) {
  nginx.log('api root: ' + api.root);
}
```

```
const exact = srv.findLocation('= /health');
const regex = srv.findLocation('~ /\.\.php$');
```

Uses the same pattern syntax as `addLocation()`.

2.3. Iterating and filtering

```
const srv = nginx.http.servers[0];

// All prefix locations with a JS handler
const jsLocs = srv.locations.filter(l =>
  l.matchType === 'prefix' && l.handler);

// All regex locations
const regexLocs = srv.locations.filter(l =>
  l.matchType === 'regex' || l.matchType === 'regex_caseless');

// Find by path prefix (e.g. all sub-locations of /api/)
const apiTree = srv.locations.filter(l =>
  l.matchType === 'prefix' && l.pattern.startsWith('/api/'));
```

3. Adding locations

3.1. `srv.addLocation(pattern [, opts])`

Adds a new location to the server and immediately rebuilds the location BST so the new location is active for all subsequent requests.

Returns the new `NginxLocation` object so properties and handler can be set in the same expression chain or immediately after.

```
const loc = srv.addLocation('/new/path');
loc.root   = '/var/www/new';
loc.handler = (req) => { req.sendFile('/index.html'); };
```

Pattern syntax:

Prefix	Example	Match type
(none)	<code>'/api/'</code>	Normal prefix — longest match wins.
<code>'= '</code>	<code>'= /ping'</code>	Exact match — wins immediately, highest priority after prefix search.
<code>'^~ '</code>	<code>'^~ /static/'</code>	Preferential prefix — disables regex scan when matched.
<code>'~ '</code>	<code>'~ /\.\.php\$'</code>	Case-sensitive PCRE regex.

Prefix	Example	Match type
'~*' '	'~* /\.\.jpg\$'	Case-insensitive PCRE regex.
'@'	'@fallback'	Named location — only reachable from <code>error_page</code> / <code>try_files</code> .

Options:

Option	Description
<code>template: '/path'</code>	Copy all config fields from an existing location (same server) as the starting point. The pattern of the new location is still <code>pattern</code> . Useful for creating near-copies of an existing location.
<code>index: N</code>	Regex locations only. Insert at position <code>N</code> in the regex check array (0 = first checked). Defaults to append (last checked).

Idempotency: if a location with the same pattern already exists, `addLocation()` returns the existing `NginxLocation` object without creating a duplicate. Safe to call multiple times.

3.2. loc.addLocation(pattern [, opts])

Delegates to the server-level `addLocation`. Available on locations obtained from `server.locations[]` or a prior `addLocation()` call. Not available on `req.location`.

```
// Equivalent – both add to the same server
srv.addLocation('/api/v2/');
apiLoc.addLocation('/api/v2/');    // apiLoc is any loc from this server
```



The BST builder automatically groups locations into a nested tree by shared prefix. You do not need to add child locations through a parent location object — adding `/api/v2/` to the server always nests it under `/api/` in the BST, regardless of which object you call `addLocation()` on.

3.3. Example: add a simple prefix location

```
// js_source: run at init_conf time

const srv = nginx.http.servers[0];
const docs = srv.addLocation('/docs/');

docs.root    = '/var/www/docs';
docs.sendfile = true;
docs.handler = (req) => {
  req.sendFile('/index.html');
};
```

3.4. Example: add an exact-match health endpoint

```
const health = srv.addLocation('= /health');
health.handler = (req) => {
  req.respond(200, { 'Content-Type': 'text/plain' }, 'OK');
};
```

Matching:

/health	—— exact match (= /health)	✓ highest priority
/health/more	—— prefix match /health	✓ (if /health exists)
/healthz	—— no match	□

3.5. Example: add a regex location at a specific position

```
const srv = nginx.http.servers[0];

// This regex will be checked BEFORE any existing regex locations
const php = srv.addLocation('~ /\.\php$', { index: 0 });
php.handler = (req) => { /* dispatch to FastCGI */ };

// This regex will be checked LAST (default append)
const fallback = srv.addLocation('~* /\.\html?$', { index: 1 });
fallback.root = '/var/www/html';
```

Regex check order (definition order):

[0]	~ /\.\php\$	← index:0 → inserted first
[1]	~* /\.\html?\$	← appended (default)
...	(existing regex locs)	

3.6. Example: add a named location for error fallback

```
const srv = nginx.http.servers[0];

const fallback = srv.addLocation('@static_fallback');
fallback.root = '/var/www/static';
fallback.handler = (req) => { req.sendFile(req.uri); };

// Wire it to 404 errors in another location
const api = srv.findLocation('/api/');
if (api) {
  api.errorPage = [{ codes: [404], uri: '@static_fallback' }];
}
```

```
}
```

3.7. Example: add a location using a template

```
const srv = nginx.http.servers[0];

// /api/v2/ gets the same config as /api/v1/ (handler, root, timeouts, ...)
// but at the new path
const v2 = srv.addLocation('/api/v2/', { template: '/api/v1/' });

// Override only what differs
v2.handler = v2Handler;
```

3.8. Flow diagram: addLocation BST rebuild

Before addLocation('/api/v2/')

Server location BST:

```
/           (prefix)
/api/       (prefix)
/api/v1/    (prefix, nested under /api/)
/static/    (prefix)
~ /\.php$   (regex[0])
```

addLocation('/api/v2/') called:

1. New clcf allocated in dyn_pool
2. entry pushed to prefix_locs[]
3. ngx_js_rebuild_loc_tree() called → new BST built from ALL prefix_locs[]
4. srv->root->static_locations = new BST root
5. New requests immediately use the new BST

After:

```
/           (prefix)
/api/       (prefix)
/api/v1/    (prefix, nested under /api/)
/api/v2/    (prefix, nested under /api/) ← NEW
/static/    (prefix)
~ /\.php$   (regex[0])
```

4. Removing locations

4.1. srv.removeLocation(pattern)

Removes the location matching **pattern** from the server. The pattern syntax is the same as

`addLocation()`. Returns `true` if found and removed, `false` if not found.

The BST is rebuilt immediately after removal so the location stops matching new requests right away.

```
const removed = srv.removeLocation('/api/v1/');
nginx.log('removed: ' + removed); // true or false
```

```
// Remove a regex location
const ok = srv.removeLocation('~ /\.\.php$');

// Remove an exact-match location
srv.removeLocation('= /health');

// Remove a named location
srv.removeLocation('@fallback');
```



`removeLocation` matches by pattern string only — the same string you passed to `addLocation`. It does not perform URI matching.

4.2. Example: maintenance mode toggle

Add a catch-all location for maintenance and remove it when done.

```
const srv = nginx.http.servers[0];
let maintenanceActive = false;

function enableMaintenance() {
  if (maintenanceActive) { return; }

  nginx.suspendAcceptance();

  const m = srv.addLocation('/'); // catches everything (longest prefix: /)
  m.handler = (req) => {
    req.respond(503, { 'Retry-After': '120' },
      'Service temporarily unavailable');
  };

  nginx.resumeAcceptance();
  maintenanceActive = true;
}

function disableMaintenance() {
  if (!maintenanceActive) { return; }

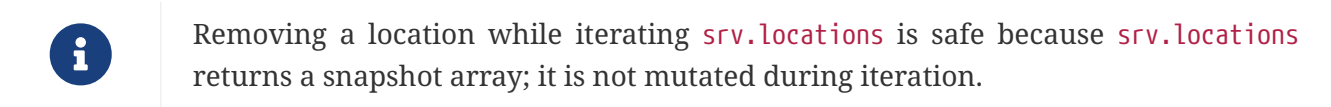
  nginx.suspendAcceptance();
  srv.removeLocation('/');
```

```
nginx.resumeAcceptance();

maintenanceActive = false;
}
```

4.3. Example: remove all dynamically added locations

```
// Remove every location whose path starts with /dynamic/  
srv.locations  
  .filter(l => l.pattern.startsWith('/dynamic/'))  
  .forEach(l => srv.removeLocation(l.path));
```



 Removing a location while iterating `srv.locations` is safe because `srv.locations` returns a snapshot array; it is not mutated during iteration.

4.4. Flow diagram: removeLocation BST rebuild

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
    |
    ├── /api/
    |   ├── /api/v1/ ← to be removed
    |   └── /api/v2/
    └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
    |
    ├── /api/
    |   └── /api/v2/
    └── /static/
```

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
    |
    ├── /api/
    |   ├── /api/v1/ ← to be removed
    |   └── /api/v2/
    └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
    |
    ├── /api/
    |   └── /api/v2/
    └── /static/
```

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
    |
    ├── /api/
    |   ├── /api/v1/ ← to be removed
    |   └── /api/v2/
    └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
    |
    ├── /api/
    |   └── /api/v2/
    └── /static/
```

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
    |
    ├── /api/
    |   ├── /api/v1/ ← to be removed
    |   └── /api/v2/
    └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
    |
    ├── /api/
    |   └── /api/v2/
    └── /static/
```

- ```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
 |
 ├── /api/
 | ├── /api/v1/ ← to be removed
 | └── /api/v2/
 └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
 |
 ├── /api/
 | └── /api/v2/
 └── /static/
```

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
 |
 ├── /api/
 | ├── /api/v1/ ← to be removed
 | └── /api/v2/
 └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

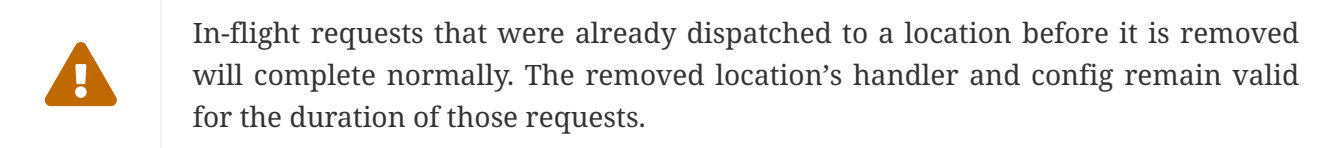
After:
BST:
 |
 ├── /api/
 | └── /api/v2/
 └── /static/
```

```
Before removeLocation('/api/v1/'):

prefix_locs[]: /, /api/, /api/v1/, /api/v2/, /static/
BST:
 |
 ├── /api/
 | ├── /api/v1/ ← to be removed
 | └── /api/v2/
 └── /static/

removeLocation('/api/v1/'):
1. entry spliced out of prefix_locs[]
2. ngx_js_rebuild_loc_tree() → new BST
3. New requests immediately stop matching /api/v1/
4. In-flight requests already dispatched to /api/v1/ complete normally

After:
BST:
 |
 ├── /api/
 | └── /api/v2/
 └── /static/
```



 In-flight requests that were already dispatched to a location before it is removed will complete normally. The removed location's handler and config remain valid for the duration of those requests.

## 5. Cloning locations

### 5.1. `loc.clone(newPattern [, opts])`

Creates a new prefix location at `newPattern` as a shallow copy of the source location — same handler, same config values (root, timeouts, proxy settings, etc.). Only plain prefix patterns (no `=`, `^~`, `~`, `@`) are supported as `newPattern`.

| Option                | Description                                                                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>depth: N</code> | Copy child locations up to <code>N</code> levels of relative nesting. <code>0</code> (default when omitted) = only the location itself. <code>Infinity</code> = copy all descendants. |

Returns the new `NginxLocation` object. Idempotent: if `newPattern` already exists as a prefix location, returns the existing wrapper.

```
const src = srv.findLocation('/api/v1/');
const dst = src.clone('/api/v2/');

// dst is an independent location – mutating it does not affect src
dst.handler = v2Handler;
```

### 5.2. Example: versioned API clone

```
const srv = nginx.http.servers[0];
const v1 = srv.findLocation('/api/v1/');

// Clone /api/v1/ tree to /api/v2/ (all immediate children)
const v2 = v1.clone('/api/v2/', { depth: 1 });

// Before: /api/v1/, /api/v1/users, /api/v1/orders
// After: all of the above PLUS:
// /api/v2/, /api/v2/users, /api/v2/orders

// Override the v2 root handler
v2.handler = (req) => {
 req.respond(200, {}, 'API v2');
};
```

Source tree (`/api/v1/`):

- `/api/v1/`
- `/api/v1/users`
- `/api/v1/orders`

After `clone('/api/v2/', { depth: 1 })`:

- `/api/v1/` (unchanged)

|                |                               |
|----------------|-------------------------------|
| /api/v1/users  | (unchanged)                   |
| /api/v1/orders | (unchanged)                   |
| /api/v2/       | (new, copy of /api/v1/)       |
| /api/v2/users  | (new, copy of /api/v1/users)  |
| /api/v2/orders | (new, copy of /api/v1/orders) |

## 5.3. Example: full subtree clone with depth:Infinity

```
const v1 = srv.findLocation('/api/v1/');

// Copies all descendants regardless of nesting depth
const v3 = v1.clone('/api/v3/', { depth: Infinity });

// Customise only the top level
v3.root = '/var/www/v3';
```

## 5.4. Example: clone a single location, no children

```
const beta = srv.findLocation('/beta/');

// depth defaults to 0: only /beta/ itself is cloned, not /beta/api/ etc.
const prod = beta.clone('/prod/');
prod.root = '/var/www/prod';
```

# 6. Cloning a server

## 6.1. srv.clone(newServerName)

Creates a shallow copy of the entire server — all locations, all config fields — and registers it as a new virtual host under `newServerName`. The clone is added to `nginx.http.servers[]` and starts accepting requests for `newServerName` immediately.

```
const primary = nginx.http.servers.find(s => s.name === 'example.com');
const mirror = primary.clone('mirror.example.com');

// Customise the clone
mirror.root = '/var/www/mirror';
```



`srv.clone()` creates independent location arrays and an independent BST for the new server. Structural changes on the clone do not affect the original, and vice versa.

## 6.2. Example: per-tenant server provisioning

```
const template = nginx.http.servers.find(s => s.name === 'template.internal');

const tenants = ['acme', 'globex', 'initech'];
tenants.forEach(tenant => {
 const srv = template.clone(tenant + '.example.com');
 srv.root = '/var/www/tenants/' + tenant;

 // Add tenant-specific admin location
 const admin = srv.addLocation('/= /admin');
 admin.handler = makeAdminHandler(tenant);
});
```

## 7. Modifying existing locations

### 7.1. Changing handler and properties

Property writes on a location obtained from `server.locations[]` or `addLocation()` use `write_mode='global'` by default — they affect all future requests on this worker immediately.

```
const loc = srv.findLocation('/api/');
if (loc) {
 loc.root = '/var/www/api-v2';
 loc.handler = newApiHandler;
}
```

### 7.2. Atomic batch modification

Use `suspendAcceptance()` when changing multiple properties to prevent any new request on this worker from seeing a half-updated location:

```
const loc = srv.findLocation('/api/');

nginx.suspendAcceptance();

loc.root = '/var/www/new';
loc.keepaliveTimeout = 75000;
loc.proxy.readTimeout = 30000;
loc.proxy.connectTimeout = 5000;

nginx.resumeAcceptance();
```

## 7.3. Rollback with snapshot

```
const loc = srv.findLocation('/api/');
const snap = loc.snapshot();

nginx.suspendAcceptance();
try {
 loc.root = cfg.root;
 loc.handler = cfg.handler;
 nginx.resumeAcceptance();
 req.json({ applied: true });
} catch (e) {
 snap.restore();
 nginx.resumeAcceptance();
 req.respond(500, {}, e.message);
}
```

## 7.4. Location swap (atomic replace)

Replace one location with another in a single suspension window:

```
async function swapLocations(srv, oldPattern, newPattern, newHandler, newRoot) {
 nginx.suspendAcceptance();

 try {
 srv.removeLocation(oldPattern);

 const loc = srv.addLocation(newPattern);
 loc.handler = newHandler;
 loc.root = newRoot;
 } finally {
 nginx.resumeAcceptance();
 }
}

// Example: swap /beta/ → /prod/
await swapLocations(srv, '/beta/', '/prod/', prodHandler, '/var/www/prod');
```

| Before swap: |          | After swap: |                     |
|--------------|----------|-------------|---------------------|
| /beta/       | (active) | /prod/      | (active, immediate) |
| /prod/       | (absent) | /beta/      | (absent)            |

## 8. Atomic location changes across all workers

### 8.1. The problem

`addLocation()`, `removeLocation()`, `clone()`, and property writes all update only the **current worker's** location BST. Other workers continue to use their own BST, which does not include the change.

To apply structural changes consistently across all workers, combine `suspendAllWorkers()` with a `SharedWorker` broadcast.

### 8.2. Pattern: broadcast structural change

```
// js_source: location_mgr.js

const cfgWorker = new SharedWorker('cfgworker.js');

cfgWorker.onmessage = (msg) => {
 const { op, pattern, opts, props } = msg.data;
 const srv = nginx.http.servers[0];

 if (op === 'add') {
 const loc = srv.addLocation(pattern, opts || {});
 if (props) {
 if (props.root) { loc.root = props.root; }
 if (props.handler) { loc.handler = eval(props.handler); } // see note
 }
 } else if (op === 'remove') {
 srv.removeLocation(pattern);
 }
};
```

```
// Request handler: add a location on all workers

loc.handler = async (req) => {
 const cfg = JSON.parse(await req.readBody());

 // Apply on this worker + broadcast to all others
 await nginx.suspendAllWorkers();

 try {
 const newLoc = srv.addLocation(cfg.pattern);
 newLoc.root = cfg.root;
 newLoc.handler = makeHandler(cfg);

 // Broadcast (other workers apply in their onmessage)
```

```

 cfgWorker.postMessage({ op: 'add', pattern: cfg.pattern,
 props: { root: cfg.root } });

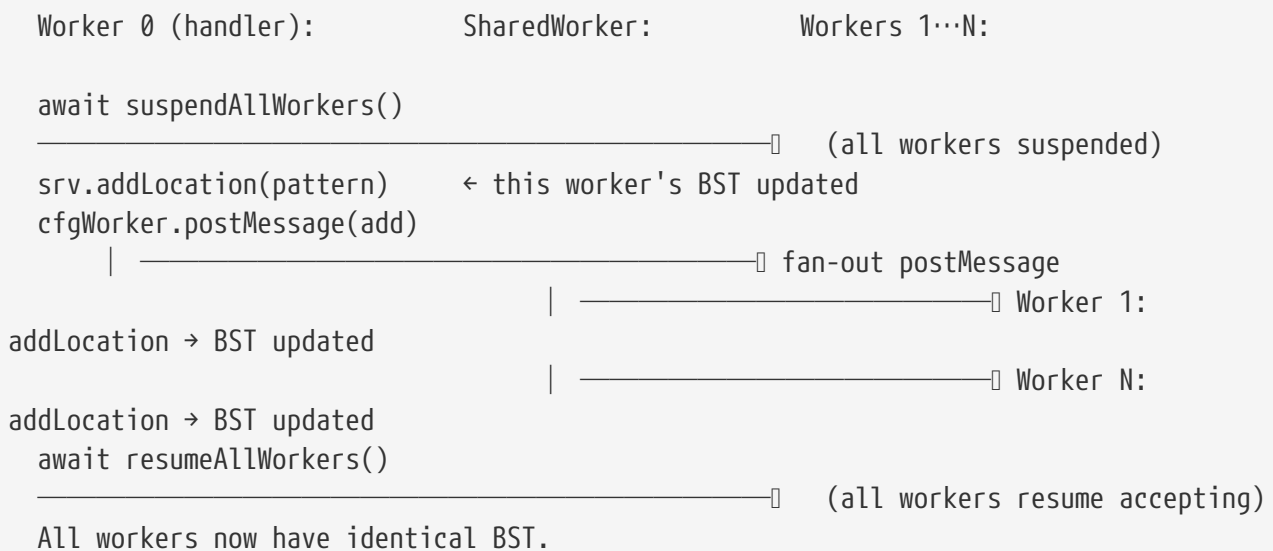
 req.json({ added: cfg.pattern });
 } finally {
 await nginx.resumeAllWorkers();
 }
};

```



JS handler functions cannot be serialised via `postMessage`. For broadcast, pass a handler identifier string and let each worker resolve it to a locally defined function (e.g. via a registry map).

## 8.3. Flow diagram: cross-worker addLocation



## 9. Safe patterns

### 9.1. Idempotent add-or-update

```

function addOrUpdate(srv, pattern, root, handler) {
 // addLocation is idempotent – returns existing loc if pattern exists
 const loc = srv.addLocation(pattern);
 loc.root = root;
 loc.handler = handler;
 return loc;
}

```

## 9.2. Temporary location (feature flag)

Add a location when a feature is enabled, remove it when disabled.

```
const FLAG_PATH = '/api/new-feature/';
let featureEnabled = false;

function setFeature(enabled) {
 if (enabled === featureEnabled) { return; }

 nginx.suspendAcceptance();

 if (enabled) {
 const loc = srv.addLocation(FLAG_PATH);
 loc.handler = newFeatureHandler;
 } else {
 srv.removeLocation(FLAG_PATH);
 }

 nginx.resumeAcceptance();
 featureEnabled = enabled;
}
```

## 9.3. Versioned API routing

```
// js_source: versions.js

const srv = nginx.http.servers[0];

// V1 is defined statically in nginx.conf
const v1 = srv.findLocation('/api/v1/');

// V2 is a clone of V1, handler overridden
const v2 = v1.clone('/api/v2/', { depth: Infinity });
v2.handler = v2ApiHandler;

// V3 is freshly added with no template
const v3 = srv.addLocation('/api/v3/');
v3.root = '/var/www/api/v3';
v3.handler = v3ApiHandler;
```

## 9.4. Location inventory endpoint

Expose the current location tree as JSON for observability:

```
const inv = srv.addLocation('= /__inventory');
```

```

inv.handler = (req) => {
 const locs = srv.locations.map(l => ({
 path: l.path,
 matchType: l.matchType,
 root: l.root,
 hasHandler: !!l.handler,
 internal: l.internal,
 }));
 req.json({ locations: locs, count: locs.length });
};

```

## 9.5. A/B routing by location clone

Route a percentage of traffic to a cloned experimental location:

```

// js_source: ab.js

const srv = nginx.http.servers[0];
const stable = srv.findLocation('/api/');
const exp = stable.clone('/api/__exp/');
exp.handler = experimentalApiHandler;

// Router: override location choice per-request
const router = srv.addLocation('/api/');
const origHandler = stable.handler;

router.handler = (req) => {
 if (Math.random() < 0.05) {
 // Internally reroute 5% to experimental handler
 experimentalApiHandler(req);
 } else {
 origHandler(req);
 }
};

```

# 10. API Reference

## 10.1. `srv.locations`

Read-only array of all `NginxLocation` objects registered on this server. Includes static (from `nginx.conf`) and dynamic (added by `addLocation()`) locations. Order: prefix/exact (longest path first), then regex (definition order), then named.

```
NginxLocation[] srv.locations
```

## 10.2. `srv.findLocation(pattern)`

Returns the `NginxLocation` for the given `pattern` string, or `null` if not found. Searches prefix/exact, regex, and named arrays.

```
NginxLocation | null srv.findLocation(pattern)
```

Pattern syntax is identical to `addLocation()`.

## 10.3. `srv.addLocation(pattern [, opts])`

Adds a new location (or returns existing if `pattern` already registered). Rebuilds the location BST immediately.

```
NginxLocation srv.addLocation(pattern)
NginxLocation srv.addLocation(pattern, { template: '/path', index: N })
```

| Throws                     | Condition                                                                    |
|----------------------------|------------------------------------------------------------------------------|
| <code>TypeError</code>     | <code>pattern</code> is not a string; regex pattern on a build without PCRE. |
| <code>InternalError</code> | BST rebuild failed (OOM).                                                    |

## 10.4. `srv.removeLocation(pattern)`

Removes the location matching `pattern` and rebuilds the BST.

```
boolean srv.removeLocation(pattern)
```

Returns `true` if found and removed, `false` if not found.

## 10.5. `srv.clone(newServerName)`

Creates a full shallow copy of the server under `newServerName`, adds it to `nginx.http.servers[]`.

```
NginxServer srv.clone(newServerName)
```

## 10.6. `loc.addLocation(pattern [, opts])`

Identical to `srv.addLocation()` — delegates to the server. Not available on `req.location`.

```
NginxLocation loc.addLocation(pattern)
```

```
NginxLocation loc.addLocation(pattern, { template: '/path', index: N })
```

## 10.7. loc.removeLocation(pattern)

Identical to `srv.removeLocation()` — delegates to the server. Not available on `req.location`.

```
boolean loc.removeLocation(pattern)
```

## 10.8. loc.clone(newPattern [, opts])

Creates a shallow copy of this location at `newPattern` (plain prefix only). Optionally copies child locations up to `depth` levels deep.

```
NginxLocation loc.clone(newPattern)
NginxLocation loc.clone(newPattern, { depth: N })
```

| depth       | Effect                                             |
|-------------|----------------------------------------------------|
| 0 (default) | Only this location; no children.                   |
| N           | Children up to N levels of relative nesting depth. |
| Infinity    | All descendants.                                   |

Throws `TypeError` if `newPattern` has a modifier prefix (`=`, `^~`, `~`, `@`).

## 10.9. loc.path, loc.matchType, loc.pattern, loc.internal

Read-only properties on every `NginxLocation`:

| Property  | Type    | Description                                                                                                                                                          |
|-----------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| path      | string  | Full pattern including modifier (e.g. <code>"= /ping"</code> , <code>"~ /\*.php\$"</code> ).                                                                         |
| matchType | string  | One of: <code>'prefix'</code> , <code>'exact'</code> , <code>'prefix_noregex'</code> , <code>'regex'</code> , <code>'regex_caseless'</code> , <code>'named'</code> . |
| pattern   | string  | Bare path/regex without modifier prefix.                                                                                                                             |
| internal  | boolean | <code>true</code> for named and internal (noname) locations.                                                                                                         |

## 10.10. nginx.suspendAcceptance() / nginx.resumeAcceptance()

Synchronous per-worker acceptance control. Use to wrap multi-step location mutations that must appear atomic to new requests on this worker. See the *for single worker* guide for full

documentation.

## 10.11. `nginx.suspendAllWorkers()` / `nginx.resumeAllWorkers()`

Async cross-worker acceptance control. Returns a `Promise<undefined>`. Use when location changes must be visible consistently across all workers before any new request is accepted. See the *for all workers* guide for full documentation.

# 11. Design notes

## 11.1. BST rebuild cost

Every `addLocation()`, `removeLocation()`, or `clone()` call rebuilds the location binary search tree from the flat `prefix_locs[]` snapshot. The rebuild is  $O(N \log N)$  in the number of prefix locations. It allocates a fresh pool for the BST nodes and releases the previous one.

For typical deployments (tens to hundreds of locations) the rebuild is measured in microseconds and is not a concern. For automated bulk operations (thousands of locations), batch them in a single suspension window to amortise the rebuild cost.

## 11.2. Dynamic pool vs static pool

Config-file locations are allocated in nginx's main memory pool (`cycle→pool`), which is never freed during a reload. Dynamically added locations are allocated in the server's `dyn_pool`, which is freed when the server wrapper is garbage-collected (typically at process exit or explicit release).

Removed locations are spliced out of the live arrays but their memory in `dyn_pool` is not reclaimed until the pool is destroyed. This is harmless for typical usage patterns.

## 11.3. `r.location` is structural read-only

`req.location` provides a per-request view of the matched location. `addLocation()`, `removeLocation()`, and `clone()` are not available on it. Field writes (`root`, `handler`, etc.) work subject to `write_mode`.

The rationale: structural changes are global operations on the server's BST; they would not make sense in a per-request isolation context.

## 11.4. Regex position and definition order

The `index` option in `addLocation('~ pattern', { index: N })` controls where the new regex is inserted in the regex array. Regex locations are always checked in array order; a lower index means higher priority.

Removing a regex location compacts the array: all locations after the removed entry shift down by one position.

## 11.5. Handler serialisation for cross-worker broadcast

JS functions cannot be transferred via `postMessage`. For cross-worker location provisioning, use a handler registry: map handler names to function references, broadcast the name, and let each worker look it up locally.

```
// Shared handler registry (defined in js_source, visible in every worker)
const handlerRegistry = {
 'api-v2': v2ApiHandler,
 'api-v3': v3ApiHandler,
 'static': staticHandler,
 'maintenance': maintenanceHandler,
};

// On receiving a broadcast:
cfgWorker.onmessage = (msg) => {
 const loc = srv.addLocation(msg.data.pattern);
 const handler = handlerRegistry[msg.data.handlerName];
 if (handler) { loc.handler = handler; }
};
```